Lab6 - Pruebas SQL

Pepe Calderón, Fernando Sola, Daniel Ayala, Inma Hernández, Margarita Cruz, Carlos Arévalo, David Ruiz

Contenido

1. Objetivo 2
2. Pruebas de aceptación de Grados 2
3. Pruebas de aceptación de Asignaturas 5
4. Pruebas de aceptación de Alumnos 10
5. Pruebas de aceptación adicionales 16

Versión PDF disponible

1. Objetivo

El objetivo de esta práctica es implementar las pruebas de aceptación del dominio del problema en SQL. El alumno aprenderá a:

- Modelar las pruebas de aceptación de la tabla Grados.
- Modelar las pruebas de aceptación de la tabla Asignaturas.
- Modelar las pruebas de aceptación de la tabla Alumnos.

2. Pruebas de aceptación de Grados

En primer lugar, definamos un procedimiento que nos permita crear un grado fácilmente y otro que nos permita actualizar un grado.

```
DELIMITER //
CREATE OR REPLACE PROCEDURE pInsertDegree(
  p_name VARCHAR(255),
  p_years INT
)
BEGIN
  INSERT INTO Degrees(name, years) VALUES
  (p_name, p_years);
END //
DELIMITER ;
```

```
DELIMITER //
CREATE OR REPLACE PROCEDURE pUpdateDegree(
  p_degreeId INT,
  p_name VARCHAR(255),
  p_years INT
)
BEGIN
  UPDATE Degrees SET
    name = p_name,
    years = p_years
  WHERE degreeId = p_degreeId;
END //
DELIMITER ;
```

A continuación, podemos implementar las pruebas de aceptación definidas en el documento de requisitos (“P” significa test positivo y “N” significa test negativo):

```
-- [P] 1. Crear grado con datos correctos.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestDegree_1()
BEGIN
  CALL pPopulateDB();
  CALL pInsertDegree('Grado en Ingeniería de la Salud', 4);
END //
DELIMITER ;

-- [N] 2. Crear grado con nombre vacío.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestDegree_2()
```

```
BEGIN
  CALL pPopulateDB();
  CALL pInsertDegree(NULL, 4);
END //
DELIMITER ;

-- [N] 3. Crear grado con el mismo nombre que otro ya existente.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestDegree_3()
BEGIN
  CALL pPopulateDB();
  CALL pInsertDegree('Grado en Ingeniería de la Salud', 4);
  CALL pInsertDegree('Grado en Ingeniería de la Salud', 4);
END //
DELIMITER ;

-- [N] 4. Crear grado con years incorrectos.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestDegree_4()
BEGIN
  CALL pPopulateDB();
  CALL pInsertDegree('Grado en Ingeniería de la Salud', -1);
END //
DELIMITER ;

-- [P] 5. Actualizar grado con datos correctos.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestDegree_5()
BEGIN
  CALL pPopulateDB();
  CALL pUpdateDegree(1, 'Grado en Ingeniería de la Salud', 3);
END //
DELIMITER ;

-- [N] 6. Actualizar grado con nombre a NULL.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestDegree_6()
BEGIN
  CALL pPopulateDB();
  CALL pUpdateDegree(1, NULL, 4);
END //
DELIMITER ;

-- [N] 7. Actualizar grado con el mismo nombre que otro existente.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestDegree_7()
BEGIN
  CALL pPopulateDB();
  CALL pUpdateDegree(1, 'Grado en Ingeniería de la Salud', 4);
  CALL pUpdateDegree(2, 'Grado en Ingeniería de la Salud', 4);
END //
```

```
DELIMITER ;

-- [N] 8. Actualizar grado con years incorrectos.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestDegree_8()
BEGIN
  CALL pPopulateDB();
  CALL pUpdateDegree(1, 'Grado en Ingeniería de Software', -1);
END //
DELIMITER ;

-- [P] 9. Eliminar grado.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestDegree_9()
BEGIN
  CALL pPopulateDB();
  CALL pInsertDegree('Grado en Ingeniería de la Salud', 4);
  DELETE FROM Degrees WHERE degreeId = 4;
END //
DELIMITER ;

-- [N] 10. Eliminar grado no existente (no lanza excepción).

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestDegree_10()
BEGIN
  CALL pPopulateDB();
  DELETE FROM Degrees WHERE degreeId = 9999;
END //
DELIMITER ;

-- [N] 11. Eliminar grado con relaciones.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestDegree_11()
BEGIN
  CALL pPopulateDB();
  DELETE FROM Degrees WHERE degreeId = 1;
END //
DELIMITER ;
```

Ejecutamos las pruebas de aceptación de Grados que hemos definido:

```
-- 1. Crear grado con datos correctos.
CALL pTestDegree_1();
-- 2. Crear grado con nombre vacío.
CALL pTestDegree_2();
-- 3. Crear grado con el mismo nombre que otro ya existente.
CALL pTestDegree_3();
-- 4. Crear grado con years incorrectos.
CALL pTestDegree_4();
-- 5. Actualizar grado con datos correctos.
CALL pTestDegree_5();
-- 6. Actualizar grado con nombre a None.
CALL pTestDegree_6();
```

```
-- 7. Actualizar grado con el mismo nombre que otro existente.
CALL pTestDegree_7();
-- 8. Actualizar grado con years incorrectos.
CALL pTestDegree_8();
-- 9. Eliminar grado.
CALL pTestDegree_9();
-- 10. Eliminar grado no existente (no lanza excepción).
CALL pTestDegree_10();
-- 11. Eliminar grado con relaciones.
CALL pTestDegree_11();
```

3. Pruebas de aceptación de Asignaturas

En primer lugar, definamos un procedimiento que nos permita crear una asignatura fácilmente y otro que nos permita actualizar una asignatura.

```
DELIMITER //
CREATE OR REPLACE PROCEDURE pInsertSubject(
  p_name VARCHAR(100),
  p_acronym VARCHAR(8),
  p_credits INT,
  p_year INT,
  p_type VARCHAR(20),
  p_degreeId INT
)
BEGIN
  INSERT INTO Subjects(name, acronym, credits, year, type, degreeId) VALUES
  (p_name, p_acronym, p_credits, p_year, p_type, p_degreeId);
END //
DELIMITER ;
```

```
DELIMITER //
CREATE OR REPLACE PROCEDURE pUpdateSubject(
  p_subjectId INT,
  p_name VARCHAR(100),
  p_acronym VARCHAR(8),
  p_credits INT,
  p_year INT,
  p_type VARCHAR(20),
  p_degreeId INT
)
BEGIN
  UPDATE Subjects SET
    name = p_name,
    acronym = p_acronym,
    credits = p_credits,
    year = p_year,
    type = p_type,
    degreeId = p_degreeId
  WHERE subjectId = p_subjectId;
END //
DELIMITER ;
```

A continuación, podemos implementar las pruebas de aceptación definidas en el documento de requisitos:

```
-- [P] 1. Crear asignatura con datos correctos.
```

```sql
DELIMITER //
CREATE OR REPLACE PROCEDURE pTestSubject_1()
BEGIN
  CALL pPopulateDB();
  CALL pInsertSubject('Física Informática', 'FI', 6, 1, 'Obligatoria', 1);
END //
DELIMITER ;

-- [N] 2. Crear asignatura con nombre vacío.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestSubject_2()
BEGIN
  CALL pPopulateDB();
  CALL pInsertSubject(NULL, 'FP', 6, 1, 'Obligatoria', 1);
END //
DELIMITER ;

-- [N] 3. Crear asignatura con el mismo nombre que otra ya existente.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestSubject_3()
BEGIN
  CALL pPopulateDB();
  CALL pInsertSubject('Fundamentos de Programación', 'FP', 6, 1, 'Obligatoria',
1);
END //
DELIMITER ;

-- [N] 4. Crear asignatura con acrónimo vacío.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestSubject_4()
BEGIN
  CALL pPopulateDB();
  CALL pInsertSubject('Física Informática', NULL, 6, 1, 'Obligatoria', 1);
END //
DELIMITER ;

-- [N] 5. Crear asignatura con el mismo acrónimo que una ya existente.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestSubject_5()
BEGIN
  CALL pPopulateDB();
  CALL pInsertSubject('Física Informática', 'FP', 6, 1, 'Obligatoria', 1);
END //
DELIMITER ;

-- [N] 6. Crear asignatura con créditos incorrectos.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestSubject_6()
BEGIN
  CALL pPopulateDB();
  CALL pInsertSubject('Física Informática', 'FI', -1, 1, 'Obligatoria', 1);
```

```
END //
DELIMITER ;

-- [N] 7. Crear asignatura con curso incorrecto.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestSubject_7()
BEGIN
  CALL pPopulateDB();
  CALL pInsertSubject('Física Informática', 'FI', 6, 7, 'Obligatoria', 1);
END //
DELIMITER ;

-- [N] 8. Crear asignatura con tipo incorrecto.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestSubject_8()
BEGIN
  CALL pPopulateDB();
  CALL pInsertSubject('Física Informática', 'FI', 6, 1, 'Incorrecto', 1);
END //
DELIMITER ;

-- [P] 9. Actualizar asignatura con valores correctos.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestSubject_9()
BEGIN
  CALL pPopulateDB();
  CALL pUpdateSubject(1, 'Física Informática', 'FI', 6, 1, 'Obligatoria', 1);
END //
DELIMITER ;

-- [N] 10. Actualizar asignatura con el mismo nombre que otra.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestSubject_10()
BEGIN
  CALL pPopulateDB();
  CALL pUpdateSubject(2, 'Fundamentos de Programación', 'FP', 6, 1, 'Obligatoria',
1);
END //
DELIMITER ;

-- [N] 11. Actualizar asignatura con nombre a None.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestSubject_11()
BEGIN
  CALL pPopulateDB();
  CALL pUpdateSubject(1, NULL, 'FP', 6, 1, 'Obligatoria', 1);
END //
DELIMITER ;

-- [N] 12. Actualizar asignatura con el mismo acrónimo que otra.
```

```
DELIMITER //
CREATE OR REPLACE PROCEDURE pTestSubject_12()
BEGIN
  CALL pPopulateDB();
  CALL pUpdateSubject(2, 'Fundamentos', 'FP', 6, 1, 'Obligatoria', 1);
END //
DELIMITER ;

-- [N] 13. Actualizar asignatura con acrónimo a None.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestSubject_13()
BEGIN
  CALL pPopulateDB();
  CALL pUpdateSubject(1, 'Fundamentos de Programación', NULL, 6, 1, 'Obligatoria',
1);
END //
DELIMITER ;

-- [N] 14. Actualizar asignatura con créditos incorrectos.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestSubject_14()
BEGIN
  CALL pPopulateDB();
  CALL pUpdateSubject(1, 'Fundamentos de Programación', 'FP', -1, 1,
'Obligatoria', 1);
END //
DELIMITER ;

-- [N] 15. Actualizar asignatura con curso incorrecto.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestSubject_15()
BEGIN
  CALL pPopulateDB();
  CALL pUpdateSubject(1, 'Fundamentos de Programación', 'FP', 6, 7, 'Obligatoria',
1);
END //
DELIMITER ;

-- [N] 16. Actualizar asignatura con tipo incorrecto.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestSubject_16()
BEGIN
  CALL pPopulateDB();
  CALL pUpdateSubject(1, 'Fundamentos de Programación', 'FP', 6, 1, 'Incorrecto',
1);
END //
DELIMITER ;

-- [P] 17. Borrar asignatura.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestSubject_17()
```

```
BEGIN
  CALL pPopulateDB();
  DELETE FROM Subjects WHERE subjectId = 3;
END //
DELIMITER ;

-- [N] 18. Borrar asignatura que ha sido borrada (no lanza excepción).

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestSubject_18()
BEGIN
  CALL pPopulateDB();
  DELETE FROM Subjects WHERE subjectId = 3;
  DELETE FROM Subjects WHERE subjectId = 3;
END //
DELIMITER ;

-- [N] 19. Borrar asignatura con relaciones.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestSubject_19()
BEGIN
  CALL pPopulateDB();
  DELETE FROM Subjects WHERE subjectId = 1;
END //
DELIMITER ;

-- [N] 20. Crear una asignación entre grado y asignatura, con grado a None.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestSubject_20()
BEGIN
  CALL pPopulateDB();
  CALL pInsertSubject('Matemática Discreta', 'MD', 6, 1, 'Obligatoria', NULL);
END //
DELIMITER ;

-- [N] 21. Actualizar una asignación entre grado y asignatura, con grado a None.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestSubject_21()
BEGIN
  CALL pPopulateDB();
  CALL pUpdateSubject(1, 'Fundamentos de Programación', 'FP', 6, 1, 'Obligatoria',
NULL);
END //
DELIMITER ;
```

Ejecutamos las pruebas de aceptación de Asignaturas que hemos definido:

```
-- 1. Crear asignatura con datos correctos.
CALL pTestSubject_1();
-- 2. Crear asignatura con nombre vacío.
CALL pTestSubject_2();
-- 3. Crear asignatura con el mismo nombre que otra ya existente.
CALL pTestSubject_3();
-- 4. Crear asignatura con acrónimo vacío.
```

```
CALL pTestSubject_4();
-- 5. Crear asignatura con el mismo acrónimo que una ya existente.
CALL pTestSubject_5();
-- 6. Crear asignatura con créditos incorrectos.
CALL pTestSubject_6();
-- 7. Crear asignatura con curso incorrecto.
CALL pTestSubject_7();
-- 8. Crear asignatura con tipo incorrecto.
CALL pTestSubject_8();
-- 9. Actualizar asignatura con valores correctos.
CALL pTestSubject_9();
-- 10. Actualizar asignatura con el mismo nombre que otra.
CALL pTestSubject_10();
-- 11. Actualizar asignatura con nombre a None.
CALL pTestSubject_11();
-- 12. Actualizar asignatura con el mismo acrónimo que otra.
CALL pTestSubject_12();
-- 13. Actualizar asignatura con acrónimo a None.
CALL pTestSubject_13();
-- 14. Actualizar asignatura con créditos incorrectos.
CALL pTestSubject_14();
-- 15. Actualizar asignatura con curso incorrecto.
CALL pTestSubject_15();
-- 16. Actualizar asignatura con tipo incorrecto.
CALL pTestSubject_16();
-- 17. Borrar asignatura.
CALL pTestSubject_17();
-- 18. Borrar asignatura que ha sido borrada (no lanza excepción).
CALL pTestSubject_18();
-- 19. Borrar asignatura con relaciones.
CALL pTestSubject_19();
-- 20. Crear una asignación entre grado y asignatura, con grado a None.
CALL pTestSubject_20();
-- 21. Actualizar una asignación entre grado y asignatura, con grado a None.
CALL pTestSubject_21();
```

4. Pruebas de aceptación de Alumnos

En primer lugar, definamos un procedimiento que nos permita crear un alumno fácilmente y otro que nos permita actualizar un alumno.

```
DELIMITER //
CREATE OR REPLACE PROCEDURE pInsertStudent(
  p_accessMethod VARCHAR(20),
  p_dni CHAR(9),
  p_firstname VARCHAR(100),
  p_surname VARCHAR(100),
  p_birthDate DATE,
  p_email VARCHAR(250),
  p_password VARCHAR(250)
)
BEGIN
  INSERT INTO Students(accessMethod, dni, firstname, surname, birthDate, email,
password) VALUES
  (p_accessMethod, p_dni, p_firstname, p_surname, p_birthDate, p_email,
p_password);
```

```
END //
DELIMITER ;

DELIMITER //
CREATE OR REPLACE PROCEDURE pUpdateStudent(
  p_studentId INT,
  p_accessMethod VARCHAR(20),
  p_dni CHAR(9),
  p_firstname VARCHAR(100),
  p_surname VARCHAR(100),
  p_birthDate DATE,
  p_email VARCHAR(250),
  p_password VARCHAR(250)
)
BEGIN
  UPDATE Students SET
  accessMethod = p_accessMethod,
  dni = p_dni,
  firstname = p_firstname,
  surname = p_surname,
  birthDate = p_birthDate,
  email = p_email,
  password = p_password
  WHERE studentId = p_studentId;
END //
DELIMITER ;
```

A continuación, podemos implementar las pruebas de aceptación definidas en el documento de requisitos:

```
-- [P] 1. Crear alumno con datos correctos.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestStudent_1()
BEGIN
  CALL pPopulateDB();
  CALL pInsertStudent('Selectividad', '12345678F', 'David', 'Ruiz', '2000-01-01',
  'david.ruiz@example.com', 'password1');
END //
DELIMITER ;

-- [N] 2. Crear alumno con el mismo DNI que otro.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestStudent_2()
BEGIN
  CALL pPopulateDB();
  CALL pInsertStudent('Selectividad', '12345678A', 'David', 'Ruiz', '2000-01-01',
  'david.ruiz@example.com', 'password2');
END //
DELIMITER ;

-- [N] 3. Crear alumno con el mismo email que otro.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestStudent_3()
BEGIN
  CALL pPopulateDB();
```

```
  CALL pInsertStudent('Selectividad', '12345678F', 'Daniel', 'Ayala',
'2000-01-01',
  'daniel@alum.us.es', 'password3');
END //
DELIMITER ;

-- [N] 4. Crear alumno con formato de fecha de nacimiento incorrecta.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestStudent_4()
BEGIN
  CALL pPopulateDB();
  CALL pInsertStudent('Selectividad', '12345678F', 'Daniel', 'Ayala',
'2000-01-32',
  'daniel.ayala@example.com', 'password4');
END //
DELIMITER ;

-- [N] 5. Crear alumno con método de acceso incorrecto.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestStudent_5()
BEGIN
  CALL pPopulateDB();
  CALL pInsertStudent('InvalidMethod', '12345678F', 'Daniel', 'Ayala',
'2000-01-01',
  'daniel.ayala@example.com', 'password5');
END //
DELIMITER ;

-- [N] 6. Crear alumno con nombre vacío.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestStudent_6()
BEGIN
  CALL pPopulateDB();
  CALL pInsertStudent('Selectividad', '12345678F', NULL, 'Ayala', '2000-01-01',
  'daniel.ayala@example.com', 'password6');
END //
DELIMITER ;

-- [N] 7. Crear alumno con apellidos vacío.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestStudent_7()
BEGIN
  CALL pPopulateDB();
  CALL pInsertStudent('Selectividad', '12345678F', 'Daniel', NULL, '2000-01-01',
  'daniel.ayala@example.com', 'password7');
END //
DELIMITER ;

-- [N] 8. Crear alumno con email vacío.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestStudent_8()
```

```sql
BEGIN
  CALL pPopulateDB();
  CALL pInsertStudent('Selectividad', '12345678F', 'Daniel', 'Ayala',
'2000-01-01', NULL,
   'password8');
END //
DELIMITER ;

-- [N] 9. Crear alumno con DNI vacío.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestStudent_9()
BEGIN
  CALL pPopulateDB();
  CALL pInsertStudent('Selectividad', NULL, 'Daniel', 'Ayala', '2000-01-01',
  'daniel.ayala@example.com', 'password9');
END //
DELIMITER ;

-- [P] 10. Actualizar alumno con datos correctos.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestStudent_10()
BEGIN
  CALL pPopulateDB();
  CALL pUpdateStudent(1, 'Selectividad', '12345678F', 'Daniel', 'Ayala',
'2000-01-01',
  'daniel.ayala@example.com', 'password10');
END //
DELIMITER ;

-- [N] 11. Actualizar alumno con el mismo DNI que otro.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestStudent_11()
BEGIN
  CALL pPopulateDB();
  CALL pUpdateStudent(1, 'Selectividad', '12345678B', 'Daniel', 'Ayala',
'2000-01-01',
  'daniel.ayala@example.com', 'password11');
END //
DELIMITER ;

-- [N] 12. Actualizar alumno con el mismo email que otro.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestStudent_12()
BEGIN
  CALL pPopulateDB();
  CALL pUpdateStudent(2, 'Selectividad', '12345678F', 'Daniel', 'Ayala',
'2000-01-01',
  'daniel@alum.us.es', 'password12');
END //
DELIMITER ;

-- [N] 13. Actualizar alumno con formato de fecha de nacimiento incorrecto.
```

```sql
DELIMITER //
CREATE OR REPLACE PROCEDURE pTestStudent_13()
BEGIN
  CALL pPopulateDB();
  CALL pUpdateStudent(1, 'Selectividad', '12345678F', 'Daniel', 'Ayala',
'2000-01-32',
  'daniel.ayala@example.com', 'password13');
END //
DELIMITER ;

-- [N] 14. Actualizar alumno con método de acceso incorrecto.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestStudent_14()
BEGIN
  CALL pPopulateDB();
  CALL pUpdateStudent(1, 'InvalidMethod', '12345678F', 'Daniel', 'Ayala',
'2000-01-01',
  'daniel.ayala@example.com', 'password14');
END //
DELIMITER ;

-- [N] 15. Actualizar alumno con nombre vacío.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestStudent_15()
BEGIN
  CALL pPopulateDB();
  CALL pUpdateStudent(1, 'Selectividad', '12345678F', NULL, 'Ayala', '2000-01-01',
  'daniel.ayala@example.com', 'password15');
END //
DELIMITER ;

-- [N] 16. Actualizar alumno con apellidos vacío.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestStudent_16()
BEGIN
  CALL pPopulateDB();
  CALL pUpdateStudent(1, 'Selectividad', '12345678F', 'Daniel', NULL,
'2000-01-01',
  'daniel.ayala@example.com', 'password16');
END //
DELIMITER ;

-- [N] 17. Actualizar alumno con email vacío.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestStudent_17()
BEGIN
  CALL pPopulateDB();
  CALL pUpdateStudent(1, 'Selectividad', '12345678F', 'Daniel', 'Ayala',
'2000-01-01', NULL,
   'password17');
END //
```

```
DELIMITER ;

-- [N] 18. Actualizar alumno con DNI vacío.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestStudent_18()
BEGIN
  CALL pPopulateDB();
  CALL pUpdateStudent(1, 'Selectividad', NULL, 'Daniel', 'Ayala', '2000-01-01',
  'daniel.ayala@example.com', 'password18');
END //
DELIMITER ;

-- [P] 19. Borrar alumno.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestStudent_19()
BEGIN
  CALL pPopulateDB();
  DELETE FROM Students WHERE studentId = 3;
END //
DELIMITER ;

-- [N] 20. Borrar alumno que no existe (no lanza excepción).

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestStudent_20()
BEGIN
  CALL pPopulateDB();
  DELETE FROM Students WHERE studentId = 999;
END //
DELIMITER ;

-- [N] 21. Borrar alumno que tiene relaciones.

DELIMITER //
CREATE OR REPLACE PROCEDURE pTestStudent_21()
BEGIN
  CALL pPopulateDB();
  DELETE FROM Students WHERE studentId = 1;
END //
DELIMITER ;
```

Ejecutamos las pruebas de aceptación de Alumnos que hemos definido:

```
-- 1. Crear alumno con datos correctos.
CALL pTestStudent_1();
-- 2. Crear alumno con el mismo DNI que otro.
CALL pTestStudent_2();
-- 3. Crear alumno con el mismo email que otro.
CALL pTestStudent_3();
-- 4. Crear alumno con formato de fecha de nacimiento incorrecto.
CALL pTestStudent_4();
-- 5. Crear alumno con método de acceso incorrecto.
CALL pTestStudent_5();
-- 6. Crear alumno con nombre vacío.
CALL pTestStudent_6();
```

```
-- 7. Crear alumno con apellidos vacío.
CALL pTestStudent_7();
-- 8. Crear alumno con email vacío.
CALL pTestStudent_8();
-- 9. Crear alumno con DNI vacío.
CALL pTestStudent_9();
-- 10. Actualizar alumno con datos correctos.
CALL pTestStudent_10();
-- 11. Actualizar alumno con el mismo DNI que otro.
CALL pTestStudent_11();
-- 12. Actualizar alumno con el mismo email que otro.
CALL pTestStudent_12();
-- 13. Actualizar alumno con formato de fecha de nacimiento incorrecto.
CALL pTestStudent_13();
-- 14. Actualizar alumno con método de acceso incorrecto.
CALL pTestStudent_14();
-- 15. Actualizar alumno con nombre vacío.
CALL pTestStudent_15();
-- 16. Actualizar alumno con apellidos vacío.
CALL pTestStudent_16();
-- 17. Actualizar alumno con email vacío.
CALL pTestStudent_17();
-- 18. Actualizar alumno con DNI vacío.
CALL pTestStudent_18();
-- 19. Borrar alumno.
CALL pTestStudent_19();
-- 20. Borrar alumno que no existe (no lanza excepción).
CALL pTestStudent_20();
-- 21. Borrar alumno que tiene relaciones.
CALL pTestStudent_21();
```

5. Pruebas de aceptación adicionales

Como ejercicio se propone diseñar e implementar las pruebas de aceptación de las tablas:

- Grupos (Groups).
- Grupos-alumnos (GroupsStudents).
- Notas (Grades).